

CS779 Report

Anuj Chaudhary — Roll No: 210167

November 2025

Abstract

With the rapid expansion of digital content in India, developing high-quality machine translation systems for regional languages has become crucial. This report presents the development and evaluation of a Neural Machine Translation (NMT) system for a mass-media conglomerate, focusing on English-to-Hindi and English-to-Bengali translation. The project was based on a bilingual corpus of nearly 150,000 parallel sentences.

A range of NMT architectures were implemented from scratch, including baseline RNN (GRU/LSTM) models, attention-based Seq2Seq models, and Transformer architectures. Different vocabulary strategies were also explored, such as simple word-level tokenization, pre-trained Word2Vec embeddings, and Byte-Pair Encoding (BPE).

Experimental results showed significant variance in translation performance. RNN-based models and models using BPE tokenization performed poorly, with BPE causing subword fragmentation and alignment issues for Hindi and Bengali scripts. The full Transformer model with BPE was unstable, producing BLEU scores as low as 0.04–0.07. In contrast, a reduced (“Half”) Transformer model using a simple word-level vocabulary achieved the strongest and most stable performance. This model consistently delivered the highest scores (0.32–0.35), demonstrating that for this dataset and language pair, preserving word boundaries was more effective than subword segmentation. The final system selection was based on this architecture, which produced the most accurate and fluent translations.

1 Introduction

With the rapid expansion of digital content in India, the demand for high-quality machine translation systems for regional languages has become increasingly important. Many existing media platforms still publish primarily in English, limiting access for non-English-speaking users. To support multilingual content creation, this project focuses on building an English-to-Hindi and English-to-Bengali Neural Machine Translation (NMT) system using a bilingual corpus of 68,849 English-Bengali and 80,797 English-Hindi sentence pairs, totaling nearly 150k parallel sentences and 299,292 sentences used during vocabulary construction.

For model development, the dataset was divided into two phases:

Training phase: ~150k pairs

Validation phase: 21,379 sentences

Testing phase: 42,757 sentences

Multiple NMT architectures were explored in the training phase, including Word2Vec + GRU, LSTM, BPE + GRU, and a partially optimized half-Transformer. These models helped analyze how recurrent and attention-based networks behave on medium-sized Indian-language datasets. However, only the Transformer-based architecture showed strong generalization and stable learning during large-scale evaluation. Therefore, the final testing phase ($\approx 42k$ sentences) was conducted exclusively using the full Transformer model, which provided the best alignment with the learned BPE vocabulary and produced the most accurate translations across both Hindi and Bengali.

2 Problem Statement

A mass-media conglomerate is expanding its publishing operations into Indian languages. The company currently produces content primarily in English, but it plans to translate its existing English articles into Bengali and Hindi. Additionally, a new team of content writers will be producing articles directly in Indian languages, further emphasizing the need for an efficient translation pipeline.

To support this transition, the Chief Technology Officer (CTO) aims to implement Neural Machine Translation (NMT) solutions. As a new member of the tech team, with prior experience from the CS779 course, you are tasked with developing a machine translation system capable of translating English text into Bengali and Hindi. To ensure fairness and promote innovation, a competition was organized within the company in which participants were required to design and implement their own NMT models from scratch.

The system must be built using an encoder-decoder architecture, enabling experimentation with different neural architectures such as RNN-based Seq2Seq models, attention-based models, and Transformer architectures. Participants are free to choose any decoding strategy (e.g., greedy decoding, beam search, or top- k sampling). However, the use of pre-trained language models is strictly prohibited; only static pre-trained word embeddings such as Word2Vec or GloVe may be employed.

3 Methodology

The methodology of this project was designed to systematically evaluate different Neural Machine Translation (NMT) architectures and preprocessing strategies for translating English to Hindi and Bengali. The focus was on understanding how vocabulary design, sequence modeling techniques, and decoding strategies influence translation quality for Indian languages. The complete workflow includes dataset preparation, vocabulary design, model architecture experiments, training procedures, and evaluation.

3.1 Dataset Preparation and Cleaning

The dataset contained approximately 150k parallel sentence pairs across two language pairs:

- English-Bengali: 68,849 pairs
- English-Hindi: 80,797 pairs

The combined corpus contained over 299,000 sentences used for vocabulary construction.

All text was normalized using NFKC Unicode normalization. English sentences were lowercased to reduce vocabulary size. Additional preprocessing included removal of non-alphanumeric symbols, compression of multiple spaces, and correction of misplaced punctuation. Sentences longer than 50 tokens were filtered out to maintain uniformity.

A validation split of 10% (21,379 sentences) was created from the training set, and final evaluation was performed on a test set of approximately 42k sentences.

3.2 Vocabulary Construction and Tokenization Strategies

Multiple vocabulary and tokenization strategies were explored:

- **Simple Word-Level Vocabulary:** Tokens were generated by splitting sentences on whitespace. Words occurring at least twice were retained. This approach preserved complete word boundaries for Hindi and Bengali, which is important for morphologically rich languages.
- **Word2Vec Embedding Vocabulary:** Pre-trained embeddings were used for English, but the lack of equivalent embeddings for Hindi/Bengali resulted in encoder-decoder embedding mismatches.
- **Byte-Pair Encoding (BPE):** Subword tokenization was used to reduce vocabulary size. However, for Hindi and Bengali, BPE often fractured meaningful character units, producing unnatural subwords and reducing fluency.

Across experiments, BPE-based vocabularies performed poorly for script-based Indian languages, while simple word-level vocabularies gave the best results.

3.3 Model Architectures Explored

The following NMT architectures were implemented and analyzed:

- **GRU/LSTM Seq2Seq Baselines:** Encoder-decoder models with GRU or LSTM cells using simple vocabularies. These struggled with long-term dependencies.
- **Word2Vec + GRU Model:** Attempted to improve encoder representations, but performance was inconsistent due to embedding mismatch.
- **BPE + GRU/LSTM Models:** Introduced subword modeling, but fragmented Hindi/Bengali tokens caused instability.
- **Attention-Based Seq2Seq (GRU + Attention):** Added global attention to improve alignment. Performance improved moderately.
- **Full Transformer + BPE:** A standard encoder-decoder Transformer model using BPE. Poor subword segmentation severely affected translation quality.
- **Reduced (“Half”) Transformer (No BPE):** Used fewer layers and a simple word-level vocabulary. Avoided BPE fragmentation and achieved the best overall translation stability and accuracy.

3.4 Training and Hyperparameters

Each model was trained independently for English–Hindi and English–Bengali using the cleaned dataset and its corresponding vocabulary. Training followed a supervised sequence-to-sequence setup, where the source sentence was encoded and the target sentence was generated token-by-token using teacher forcing.

Hyperparameter Settings

- **Batch Size:** 64–128; the final Transformer used 128.
- **Learning Rate:** 0.001 for GRU/LSTM models; 1×10^{-4} for Transformer models.
- **Optimizer:** Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$ for Transformer).
- **Weight Initialization:** Embeddings initialized with a normal distribution (std = 0.02); linear layers with Xavier uniform initialization.
- **Gradient Clipping:** Threshold of 0.5–1.0.
- **Epochs:** RNN models converged in 7–12 epochs; transformer models trained up to 20 epochs.
- **Early Stopping:** Triggered after 3–4 epochs without validation improvement.

Mixed Precision Training

To improve training speed, automatic mixed precision (AMP) using `torch.cuda.amp.autocast()` and `GradScaler` was applied for Transformer models.

Regularization

- Dropout: 0.2–0.35 across embeddings, RNN layers, attention layers, and feed-forward blocks.
- Label smoothing: 0.1 for Transformer variants.

Sequence Constraints

- Maximum sequence length: 50 tokens
- Special tokens: `<SOS>`, `<EOS>`, `<UNK>`, `<PAD>`
- Padding masks were applied during attention computations.

4 Experimental Results and Analysis

Multiple architectures and preprocessing strategies were evaluated. The performance of each model, along with the decoding strategies used, is summarized below. The “score” refers to a composite evaluation metric (e.g., BLEU or ChrF) that reflects overall translation quality.

4.1 Simple Vocabulary + GRU/LSTM Seq2Seq (Baseline Models)

Score: ~ 0.01

Decoding: Greedy

Analysis: These baseline models struggled to capture long-distance dependencies and the morphological richness of Hindi and Bengali. Greedy decoding further restricted output diversity. The extremely low scores confirmed that classical RNN-based Seq2Seq architectures were insufficient for this task.

4.2 Word2Vec Embeddings + GRU

Score: 0.13 (sometimes dropping to 0.02)

Decoding: Greedy

Analysis: The mismatch between English Word2Vec embeddings and randomly initialized Hindi/Bengali embeddings caused encoder-decoder alignment inconsistencies. As a result, the model exhibited unstable and unreliable translation behavior.

4.3 Byte-Pair Encoding (BPE) + GRU/LSTM

Score: Typically around 0.10, occasionally peaking at 0.29

Decoding: Greedy

Analysis: BPE generated fragmented and unnatural subword units for Hindi and Bengali scripts. Greedy decoding amplified these issues by producing incomplete, repetitive, or semantically broken sequences.

4.4 Attention-Based Seq2Seq (GRU + Attention)

Score: Approximately 0.17

Decoding: Greedy, k -sampling, n-gram sampling

Analysis: The addition of attention improved alignment between source and target tokens and moderately enhanced translation fluency. Although decoding variations gave minor improvements, the overall performance remained significantly below transformer-based architectures.

4.5 Full Transformer + BPE

Score: Typically 0.04, occasionally rising to 0.07 or 0.20

Decoding: Greedy batch decoding, k -sampling

Analysis: Subword fragmentation caused by BPE led to severe alignment problems in the Transformer. Models frequently produced unnatural tokens, repeated segments, and unstable sequences. Most evaluations remained in the 0.04–0.07 range, and training times were considerably higher than RNN-based models.

4.6 Reduced (“Half”) Transformer (No BPE, Simple Vocabulary)

Score: 0.32–0.35

Decoding: Greedy decoding, Beam search

Analysis: This model consistently achieved the strongest and most stable performance. Eliminating BPE-preserved word boundaries is crucial for Hindi and Bengali. Despite using fewer layers, the reduced Transformer captured long-range dependencies effectively. Beam search further improved fluency, making this the best-performing configuration overall.

4.7 Final Model Selection

Based on comprehensive experimentation, the Reduced (“Half”) Transformer with simple word-level vocabulary was selected as the final model. It consistently achieved the highest translation scores (0.32–0.35), avoided BPE-induced segmentation errors, and produced the most fluent translations when combined with beam search.

4.8 Final Model Architecture

The final model selected for the testing phase was a reduced (“Half”) Transformer architecture. Despite having fewer layers than the full Transformer, it consistently outperformed all other configurations due to better stability, cleaner word-level alignment, and significantly improved performance on Hindi and Bengali script-based translations.

4.9 Testing Phase Results

The final testing phase evaluated multiple submitted models. Table 2 summarizes the scores obtained across all submissions.

5 Challenges and Discussion

Several key challenges were identified during the project, primarily related to tokenization and evaluation for Indian languages.

- **Tokenizer and Script Mismatch:** Hindi and Bengali scripts do not segment cleanly with BPE. This resulted in over-segmentation, vocabulary inflation, and poor alignment between English and target languages. Consequently, the Full Transformer + BPE model struggled and produced low scores (0.07–0.20).
- **Evaluation Score Instability:** BLEU frequently returned scores of zero due to its strict requirement for exact word matches. Minor segmentation or script variations caused full mismatches even when translations were partially correct. Character-based metrics such as ChrF (typically ~ 0.20) revealed that BLEU is unreliable for evaluating Indian-language NMT systems.
- **Model Complexity vs. Dataset Size:** Larger models, such as the Full Transformer, tended to overfit or converge slowly on the $\sim 150\text{k}$ -pair dataset. In contrast, lighter models—particularly the simple vocabulary + Reduced Transformer—achieved

Table 1: Final Transformer Architecture Used in the Second Training Pipeline

Component	Configuration
Model Type	Full Transformer (Encoder–Decoder)
Embedding Dimension	384
Number of Encoder Layers	4
Number of Decoder Layers	4
Number of Attention Heads	8
Feed-Forward Network Size	1024
Dropout	0.3
Maximum Sequence Length	45 tokens
Vocabulary Size (EN)	63,451
Vocabulary Size (BN)	41,129
Vocabulary Size (HI)	36,547
Tokenization Strategy	Simple Word-Level (No BPE)
Special Tokens	<PAD>, <SOS>, <EOS>, <UNK>
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay = $1e-4$)
Learning Rate Schedule	Noam Warmup (step-based)
Initial Learning Rate	5×10^{-4}
Batch Size	64
Loss Function	Cross-Entropy + Label Smoothing (0.1)
Gradient Clipping	1.0
Mixed Precision	Enabled (AMP)
Decoding Strategy	Batched Greedy Decoding
Vocabulary Saving	JSON export for EN/BN/HI
Training Epochs	30 (Early Stopping = 3)

Table 2: Testing Phase Results for Submitted Models

ID #	Date	Score
416136	2025-11-06 23:52	0.23
416003	2025-11-06 23:01	0.24
415486	2025-11-06 18:39	0.24
415358	2025-11-06 16:15	0.18
414927	2025-11-06 04:27	0.08
414882	2025-11-06 03:16	0.31
414517	2025-11-05 20:56	0.33

significantly better performance (0.32–0.35). This demonstrates that heavier architectures were not optimal for a medium-sized dataset.

6 Conclusion

The testing phase compared two primary model configurations: (1) a full transformer using BPE subword tokenization and (2) a transformer using a simple, frequency-based vocabulary without BPE. Both models were trained for up to 30 epochs to thoroughly evaluate their learning behavior on English-Hindi and English-Bengali parallel datasets.

The experimental results showed a clear and consistent pattern. The Transformer with simple vocabulary achieved the strongest performance, producing the highest evaluation scores (0.31–0.33) and maintaining stable behavior across large-scale testing. Its superior performance stemmed from cleaner word-level alignment, fewer segmentation errors, and better compatibility with Hindi and Bengali morphology.

In contrast, the Transformer with BPE tokenization produced unstable results, typically scoring between 0.08 and 0.24. Subword fragmentation in Hindi and Bengali led to poor lexical alignment and frequent BLEU failures. Even after 30 epochs of training, the BPE-based Transformer could not match the performance of the simple-vocabulary model.

Overall, the testing-phase results confirm that a simple vocabulary combined with a Transformer architecture is the most effective and reliable approach for this dataset. This configuration delivers cleaner decoding, stronger generalization, and consistently higher evaluation scores than the BPE-based alternative, despite using the same training schedule and architectural depth.

7 References

1. Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
2. Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
3. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
4. Koehn, P. (2017). Neural Machine Translation. *arXiv preprint arXiv:1703.01619*.
5. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., *et al.* (2019). PyTorch: An imperative-style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
6. Sennrich, R., Haddow, B., & Birch, A. (2015). Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*.
7. Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, 27.
8. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.